

Spring 2026, Digital Computer Concepts and Practice Term Project Specification

Language and Data Intelligence Lab, SNU

Task

- Implement an efficient Wordle puzzle solver that minimizes the number of queries needed to identify and submit the secret word, under structured stochastic feedback noise.
- Implement a simple HTTP server that returns either a guess or a final submission upon request.
- Create a problem instance (secret word + candidate word list).

Matching Rules

Given an answer (secret word) and a query, the feedback code is computed as follows:

1. If the two words have the same letter at the i -th ($1 \leq i \leq 5$) position, the corresponding result is 2.
2. If the query contains letters not present in the answer, they are marked as 0.
3. If the query contains matching but misplaced letters, they are marked as either 1 or 0, such that:
 - The total number of 2's and 1's assigned to a character cannot exceed its count in the answer.
 - All 0's, if any, must appear after all 1's for that character.

For example:

```
ANSWER TWEET
QUERY MELEE
-----
RESULT 01020
```

Note how the three E's in the query are handled. The second "E" is marked as 2 since it matches the correct position. The other two "E"'s are assigned 1 and 0 from left to right, reflecting that only two "E"'s exist in the answer.

Your solver will not see these numeric codes directly. Instead, the grader will provide a deterministic rule-based English description of the feedback. You can parse this string mechanically. The verbalizing rules are:

```

1 def rule_verbalize(guess, feedback):
2     names = ["first", "second", "third", "fourth", "fifth"]
3     parts = []
4     for name, ch, mark in zip(names, guess, feedback):
5         if mark == "2":
6             parts.append(f"The {name} letter '{ch}' is correct.")
7         elif mark == "1":
8             parts.append(f"The {name} letter '{ch}' is misplaced.")
9         else:
10            parts.append(f"The {name} letter '{ch}' is absent.")
11    return "\n".join(parts)

```

Example feedback string:

```

>>> print(rule_verbalize("melee", "01020"))
The first letter 'm' is absent.
The second letter 'e' is misplaced.
The third letter 'l' is absent.
The fourth letter 'e' is correct.
The fifth letter 'e' is absent.

```

Noisy Feedback

The feedback returned to your solver may be noisy. On each guess, the grader first determines an *effective secret* and computes feedback against that effective secret.

- With probability $1 - p - q$, the effective secret is the true secret word.
- With probability p , the effective secret is obtained by changing exactly one randomly chosen letter of the true secret to a different lowercase letter.
- With probability q , the effective secret is obtained by changing exactly two randomly chosen positions of the true secret, each to a different lowercase letter.

The perturbed effective secret does not need to be a valid English word or a member of the candidate list.

To ensure that early noisy feedback is represented, the first three guess turns are guaranteed to contain at least one noisy feedback event. Concretely, the grader first samples the feedback type for each turn according to the probabilities above. If none of the first three turns is noisy, one of those three turns is selected uniformly at random and forced to be noisy. The forced noisy turn is one-letter or two-letter noise in proportion to $p : q$; if $p = q = 0$, it defaults to one-letter noise.

For reproducibility and fairness, the grader precomputes the noisy turns and perturbed effective secrets for every problem and turn from a fixed seed. Thus the same problem, seed, and noise parameters produce the same feedback process for every team, regardless of how many turns a particular solver takes before submitting.

Problem Setup

Each problem has a unique problem ID. For each problem in the test suite, the grader will first provide the list of candidate words associated with that problem. All words (both secret word and candidates) will be lowercase English words of length 5.

The interaction between the grader and your server follows a simple HTTP-based protocol. Your server must handle two kinds of requests.

1. Start Problem Request

At the start of each problem, the grader will send:

```
POST /start_problem
Content-Type: application/json
{
  "problem_id": "<problem_id>",
  "candidate_words": ["word1", "word2", ..., "wordN"],
  "noise_probability": 0.33,
  "two_letter_noise_probability": 0.33,
  "force_noise_turns": 3,
  "max_turns": 100,
  "guess_time_budget": 60
}
```

Your server must respond to this request with HTTP 200 OK and an empty JSON object:

```
HTTP 200 OK
Content-Type: application/json
{}
```

The grader will not use the content of this response, but failure to respond within the time limit (5 seconds) will be treated as a timeout for that problem.

Your server must store both **problem_id** and the provided candidate words. Your guesses must only select words from this candidate list. The noise and budget fields are provided so that your solver can take the known feedback process into account.

2. Next Guess Request

Then, the grader will repeatedly ask for your next action. Each request includes only the feedback for your previous guess:

```
POST /act
Content-Type: application/json
```

```
{
  "problem_id": "<problem_id>",
  "feedback": "<feedback_for_previous_guess>",
  "turn": <turn_number>
}
```

- **feedback** contains the feedback for your previous guess.
- On the first turn, **feedback** will be **null**.
- **feedback** will be a deterministic rule-based English description of the previous feedback.
- **turn** is the current turn number (starting from 1).

Only the most recent feedback will be provided. Your server must maintain complete feedback history internally for each problem.

3. Action Response

For each `/act` request, your server must respond with either a **guess** action or a **submit** action.

A guess action asks for another noisy feedback observation:

```
HTTP 200 OK
Content-Type: application/json
{
  "action": "guess",
  "word": "<your_guess>"
}
```

A submit action commits to the final answer:

```
HTTP 200 OK
Content-Type: application/json
{
  "action": "submit",
  "word": "<your_final_answer>"
}
```

- Your word must always be one of the candidate words provided.
- An invalid guess will result in immediate failure for that problem.
- Submitting the correct secret solves the problem.
- Submitting an incorrect secret incurs a severe penalty; do not submit unless your solver is confident.

Additional Notes

- Only one problem will be active at a time; concurrent handling of multiple problems is not required.
- You are responsible for all internal state management.
- All necessary information is provided through these HTTP requests; no external files can be consulted.
- The feedback is generated according to the matching and noise rules above.
- The official feedback format is deterministic and can be parsed without using an LLM.
- Your server must read the port number from the environment variable `PORT`, defaulting to 8000 if not set. During grading, your server may be assigned a different port.

Starter Code

Starter code is provided on eTL:

- `grader.py`: Simplified grader for your own testing. It provides the same deterministic rule-based feedback format as the official grader.
- `team00_problem.json`: A small example problem instance.
- `team00.py`: Starter code template for your solver and HTTP server. It contains only a minimal baseline; you must implement the core solver logic. Modify the team number to match yours.

You may restructure your code as needed, but your server must be runnable using:

```
python team00.py
```

Budget

- Your server must be fully ready to receive requests within 5 seconds after startup. That is, the grader waits for 5 seconds before issuing a `/start_problem` request.
- Then, your server has 5 seconds to handle the `/start_problem` request.
- After that, your server has a total of 60 seconds for all `/act` requests for that problem combined.
- The grader starts a fresh solver process for each problem. If your solver exceeds the time budget, that problem is marked as a timeout and the process may be terminated.

Writing a Problem

Create one Wordle problem instance consisting of a candidate word list and a secret word. The candidate words must be selected from the provided global list of valid

lowercase 5-character English words. Your solver will be tested on your own problem instance as well, so ensure it can solve it efficiently.

Submission

Submit the following on eTL:

- Presentation slides (replacing a separate written report).
- Full code (all files necessary to run your server) as a `zip` file. It should not exceed 1 MB in size.
- Your problem instance as `team00_problem.json`, containing a JSON object with `secret_word` and `candidate_words` keys.

```
{
  "secret_word": "hello",
  "candidate_words": [
    "hello",
    "world"
  ]
}
```

Submission penalties. Submissions may be penalized for issues including, but not limited to, the following:

- Late submission: A late submission will receive a 20% deduction from the total project grade. Submissions more than 12 hours after the deadline will not be accepted.
- Incorrect file names or formats: If the issue is minor and can be resolved by a simple administrative fix, such as renaming a file or correcting an accidental file extension, TAs may apply the fix with a 10% deduction. Otherwise, the affected file will be treated as missing from the submission.
- Failure to run: If the submitted code cannot be executed by the grader, the team will receive zero credit for the performance score.
- Oversized submission: Submissions exceeding the size limit will be rejected.

Evaluation

The solver will be evaluated on the same problem set under three noise tracks:

- Track 1: $p = 0.0$, $q = 0.0$, but with one forced one-letter noisy feedback injected within the first three turns.
- Track 2: $p = 0.33$, $q = 0.33$.
- Track 3: $p = 0.33$, $q = 0.66$.

For each track, all teams will solve the same problems under the same deterministic noise schedule. Guess actions and the final submit action both count as turns.

Let $C = 100$ be the per-problem cap. For each problem:

- If the solver correctly submits the secret in t turns ($t < C$), the score-turn count is t .
- If the solver fails to solve the problem for any reason, the score-turn count is C .
- Failure cases include timeout, server crash, malformed response, invalid action, invalid word, reaching the maximum number of turns, and submitting an incorrect secret.

Let x be a team's average score-turn count on a track, and let y be the best average score-turn count achieved by any team on that track. The track score is:

$$\max\left(0, 100 - 20 \log_2\left(\frac{x}{y}\right)\right).$$

Thus, if the best team averages $y = 5$ turns and another team averages $x = 10$ turns, the latter receives 80 points for that track.

Final evaluation will consider:

- The three track scores above.
- Validity of the submitted problem instance.
- Presentation quality.

The submitted problem instance will be graded as a binary component. A team receives full credit for this component if it submits exactly one valid problem instance in the specified format. The instance must contain a valid `secret_word` and a valid `candidate_words` list, and the secret word must be included in the candidate list. If the problem instance is missing, malformed, contains invalid words, contains multiple problems, or does not follow the required JSON schema, the team receives zero credit for this component.

Presentation

Each team will present for 5 minutes (mandatory). Explain your method, design decisions, challenges, and how you addressed them.

Schedule

- Code and slides submission: Due 6/14 11:59 PM
- Presentations: 6/15

Final Note

The main focus of this project is on problem-solving logic. In this sense, you are kindly asked to avoid unnecessary third-party libraries.

If you believe that a particular package is essential for your solution, you must provide the TAs with the necessary means to reproduce your environment, such as a complete `requirements.txt` file. However, please note that any such submission will be handled on a *best-effort* basis; we cannot guarantee that we will be able to reproduce or resolve all external dependencies during grading.

Also, your submission must run entirely from the submitted Python source code. Pre-compiled binaries, external executables, shared libraries, or dynamically downloaded components are not allowed. Any preprocessing required by your method must be performed at runtime after the grader starts your solver process. Submissions must not rely on precomputed hidden artifacts, offline-generated lookup tables, or serialized solver states that are not clearly reproducible from the submitted source code.